

BÁO CÁO CUỐI KÌ**Câu 1: Thiết kế một hệ thống xử lý tính toán sử dụng cấu trúc FSM****Mã nguồn**

```

module FSM_Top (
    input wire CLOCK_50, // Đầu vào là xung clock hệ thống 50 MHz
    input wire [8:0] SW,   // Đầu vào là các công tắc gạt 9-bit (SW0-SW8)
    input wire [2:0] KEY,  // Đầu vào là các nút nhấn (KEY0-KEY2)
    output wire [9:0] LEDR, // Đầu ra là các đèn LED trên board (LEDR0-LEDR9)
    output wire [6:0] HEX0, // Đầu ra là LED 7 đoạn HEX0
    output wire [6:0] HEX1, // Đầu ra là LED 7 đoạn HEX1
    output wire [6:0] HEX2, // Đầu ra là LED 7 đoạn HEX2
    output wire [6:0] HEX3, // Đầu ra là LED 7 đoạn HEX3
    output wire [6:0] HEX4, // Đầu ra là LED 7 đoạn HEX4
    output wire [6:0] HEX5  // Đầu ra là LED 7 đoạn HEX5
);

wire w_led; // tín hiệu xung chậm từ Clock_Divider
wire w_key1_out; // tín hiệu KEY1 đã qua chống dội
wire w_key2_out; // tín hiệu KEY2 đã qua chống dội
wire [1:0] w_state; // chứa trạng thái FSM hiện tại
wire [3:0] w_a; // dữ liệu A
wire [3:0] w_b; // dữ liệu B
wire [3:0] w_result; // kết quả tính toán
wire w_op; // chứa lựa chọn phép toán (OP)
wire w_valid; // cờ báo kết quả hợp lệ
wire w_st4_result; // cờ báo trạng thái ST4 đã xong

Clock_Divider u_clk (
    .CLOCK_50(CLOCK_50), // Kết nối xung hệ thống 50 MHz
    .KEY(KEY[0]), // Dùng KEY0 làm Reset cho bộ chia xung
    .Led(w_led) // Đầu ra xung chậm nối vào w_led
);

DEBOUNCER u_btn (
    .CLOCK_50(CLOCK_50), // Kết nối xung hệ thống 50 MHz
    .KEY(KEY[0]), // Dùng KEY0 làm Reset cho bộ chống dội
    .Key1_in(KEY[1]), // Nhận tín hiệu KEY1
    .Key2_in(KEY[2]), // Nhận tín hiệu KEY2
    .Key1_out(w_key1_out), // Tín hiệu sạch KEY1
    .Key2_out(w_key2_out) // Tín hiệu sạch KEY2
);

FSM_Controller u_fsm (
    .CLOCK_50(CLOCK_50), // Kết nối xung hệ thống 50 MHz
    .KEY(KEY[0]), // Dùng KEY0 làm Reset cho FSM
    .Key1_out(w_key1_out), // Tín hiệu sạch KEY1
    .Key2_out(w_key2_out), // Tín hiệu sạch KEY2
    .SW_A(SW[3:0]), // Gán SW[3:0] làm đầu vào A
    .SW_B(SW[7:4]), // Gán SW[7:4] làm đầu vào B
    .SW_OP(SW[8]), // Gán SW[8] làm OP
    .state(w_state), // Kết nối trạng thái FSM

```

```

.A(w_a),      // Kết nối dữ liệu A
.B(w_b),      // Kết nối dữ liệu B
.result(w_result), // Kết nối kết quả
.OP(w_op),    // Kết nối toán tử OP
.valid(w_valid), // Kết nối cờ valid
.ST4_result(w_st4_result) // Kết nối cờ ST4_result
);

LED_7 u_disp (
.CLOCK_50(CLOCK_50), // Kết nối xung hệ thống 50 MHz
.state(w_state),     // Nhận trạng thái FSM
.Led(w_led),         // Nhận tín hiệu Led
.OP(w_op),           // Nhận tín hiệu OP
.result_valid(w_valid), // Nhận tín hiệu valid
.st4_result(w_st4_result), // Nhận tín hiệu ST4_result
.data_a(w_a),        // Nhận dữ liệu A
.data_b(w_b),        // Nhận dữ liệu B
.result(w_result),    // Nhận kết quả
.HEX0(HEX0), .HEX1(HEX1), .HEX2(HEX2), .HEX3(HEX3), .HEX4(HEX4), .HEX5(HEX5) // Gán đầu ra cho 6 LED
7 đoạn
);
assign LEDR = {10{w_led}}; // Gán 10 LED sáng/tắt theo xung
endmodule

```

```

module Clock_Divider (
input wire CLOCK_50, // Đầu vào là xung clock hệ thống 50 MHz
input wire [0:0] KEY, // Tín hiệu Reset tích cực mức thấp (sử dụng KEY[0])
output reg Led // Đầu ra là các trạng thái của Led
);
reg [24:0] counter; // Bộ đếm 25 bit dùng để tạo trễ thời gian

// Khởi logic thực thi tại cạnh lên của CLOCK_50 hoặc khi có reset (tín hiệu KEY[0])
always @(posedge CLOCK_50 or negedge KEY[0]) begin
if (!KEY[0]) begin // Kiểm tra tín hiệu reset (mức thấp từ KEY[0])
counter <= 25'd0; // Đặt bộ đếm về 0
Led <= 1'b0; // Đặt Led về trạng thái tắt
end else if (counter == 25'd24_999_999) begin // Kiểm tra nếu đã đếm đủ 0.5 giây
counter <= 25'd0; // Reset bộ đếm sau khi đạt ngưỡng
Led <= ~Led; // Đảo trạng thái Led
end else begin
counter <= counter + 1'b1; // Tăng bộ đếm lên 1 đơn vị
end
end
endmodule

```

```

module DEBOUNCER (
input wire CLOCK_50, // Đầu vào là xung clock hệ thống 50 MHz
input wire [0:0] KEY, // Tín hiệu reset tích cực mức thấp (dùng KEY[0])
input wire Key1_in, Key2_in, // Tín hiệu đầu vào từ 2 nút nhấn
output wire Key1_out, Key2_out // Tín hiệu đầu ra sau khi đã chống dội nút
);
// Biến lưu trữ trạng thái trung gian để đồng bộ tín hiệu cho Key 1
reg k1_s0, k1_s1, k1_old;
// Biến lưu trữ trạng thái trung gian để đồng bộ tín hiệu cho Key 2

```

```

reg k2_s0, k2_s1, k2_old;

// Gán giá trị đầu ra: phát hiện cạnh xuống (khi nhấn nút từ 1 xuống 0) cho Key 1
assign Key1_out = k1_old & ~k1_s1;
// Gán giá trị đầu ra: phát hiện cạnh xuống (khi nhấn nút từ 1 xuống 0) cho Key 2
assign Key2_out = k2_old & ~k2_s1;

// Khởi logic thực thi tại cạnh lên của xung clock hoặc khi có tín hiệu reset từ KEY[0]
always @(posedge CLOCK_50 or negedge KEY[0]) begin
    if (!KEY[0]) begin // Kiểm tra nếu tín hiệu reset KEY[0] đang ở mức thấp
        // Đưa tất cả các biến lưu trạng thái về trạng thái không nhấn (mức 1)
        k1_s0 <= 1'b1; k1_s1 <= 1'b1; k1_old <= 1'b1;
        k2_s0 <= 1'b1; k2_s1 <= 1'b1; k2_old <= 1'b1;

    end else begin // Nếu không có reset, thực hiện lấy mẫu tín hiệu
        // Cập nhật trạng thái tuần tự cho Key 1 để đồng bộ
        k1_s0 <= Key1_in; k1_s1 <= k1_s0; k1_old <= k1_s1;
        // Cập nhật trạng thái tuần tự cho Key 2 để đồng bộ
        k2_s0 <= Key2_in; k2_s1 <= k2_s0; k2_old <= k2_s1;
    end
end
endmodule

```

```

module FSM_Controller (
    input wire CLOCK_50, // Đầu vào là xung clock hệ thống 50 MHz
    input wire [0:0] KEY, // Tín hiệu reset tích cực mức thấp (dùng KEY[0])
    input wire Key1_out, // Tín hiệu nút nhấn 1 đã chống dội
    input wire Key2_out, // Tín hiệu nút nhấn 2 đã chống dội
    input wire [3:0] SW_A, // Đầu vào cụ thể cho A (SW[3:0])
    input wire [3:0] SW_B, // Đầu vào cụ thể cho B (SW[7:4])
    input wire SW_OP, // Đầu vào cụ thể cho OP (SW[8])
    output reg [1:0] state, // Trạng thái hiện tại của FSM
    output reg [3:0] A, // Giá trị 4 bit A
    output reg [3:0] B, // Giá trị 4 bit B
    output reg [3:0] result, // Kết quả 4-bit
    output reg OP, // OP (phép toán)
    output reg valid, // valid (cờ kết quả)
    output reg ST4_result // ST4_result (cờ tính kết quả ST4)
);
// Định nghĩa các hằng số trạng thái
localparam [1:0] ST1 = 2'b00; // Trạng thái khởi đầu
localparam [1:0] ST2 = 2'b01; // Trạng thái nạp dữ liệu
localparam [1:0] ST3 = 2'b10; // Trạng thái chọn phép toán
localparam [1:0] ST4 = 2'b11; // Trạng thái thực hiện tính toán

// Khởi logic thực thi tại cạnh lên của CLOCK_50 hoặc khi có reset từ KEY[0]
always @(posedge CLOCK_50 or negedge KEY[0]) begin
    if (!KEY[0]) begin // Kiểm tra tín hiệu reset hệ thống từ KEY[0]
        state <= ST1; // Đưa trạng thái về ST1
        A <= 0; // Reset giá trị A
        B <= 0; // Reset giá trị B
        OP <= 0; // Reset toán tử
        result <= 0; // Reset kết quả
        valid <= 0; // Reset cờ valid
    end
end

```

```

ST4_result <= 0; // Reset cờ ST4_result
end else begin
  case (state) // Kiểm tra trạng thái hiện tại
    ST1: if (Key1_out) state <= ST2; // Nếu nhấn Key1, chuyển sang ST2

    ST2: begin // Trạng thái nạp dữ liệu
      if (Key2_out) begin A <= SW_A; B <= SW_B; end // Lưu dữ liệu từ SW_A, SW_B
      if (Key1_out) state <= ST3; // Nếu nhấn Key1, chuyển sang ST3
    end

    ST3: begin // Trạng thái chọn phép toán
      if (Key2_out) OP <= SW_OP; // Nếu nhấn Key2, lưu phép toán từ SW_OP
      if (Key1_out) begin state <= ST4; ST4_result <= 0; end // Nếu nhấn Key1, chuyển sang ST4
    end

    ST4: begin // Trạng thái thực hiện tính toán
      if (Key2_out) begin // Nếu nhấn Key2, thực hiện tính toán
        result <= (OP == 0) ? (A + B) : (A - B); // Phép cộng hoặc trừ
        valid <= 1; ST4_result <= 1; // Cập nhật các cờ kết quả
      end
      if (Key1_out) state <= ST1; // Nếu nhấn Key1, quay lại ST1
    end
  endcase
end
end
endmodule

```

```

module LED_7 (
  input wire CLOCK_50, // Đầu vào là xung clock hệ thống 50 MHz
  input wire [1:0] state, // Trạng thái hiện tại của FSM
  input wire Led, // Tín hiệu Led tạo xung 1Hz hoặc 0.5 Hz
  input wire OP, // Tín hiệu lựa chọn phép toán (Operation)
  input wire result_valid, // Cờ báo hiệu kết quả hợp lệ
  input wire st4_result, // Cờ báo hiệu đã tính toán xong ở ST4
  input wire [3:0] data_a, // Giá trị dữ liệu A
  input wire [3:0] data_b, // Giá trị dữ liệu B
  input wire [3:0] result, // Giá trị kết quả tính toán
  output reg [6:0] HEX0, // Đầu ra LED 7 đoạn HEX0
  output reg [6:0] HEX1, // Đầu ra LED 7 đoạn HEX1
  output reg [6:0] HEX2, // Đầu ra LED 7 đoạn HEX2
  output reg [6:0] HEX3, // Đầu ra LED 7 đoạn HEX3
  output reg [6:0] HEX4, // Đầu ra LED 7 đoạn HEX4
  output reg [6:0] HEX5 // Đầu ra LED 7 đoạn HEX5
);

```

```

function [6:0] display7; // Hàm giải mã giá trị 4-bit sang mã 7-đoạn
  input [4:0] char; // Đầu vào là ký tự cần hiển thị
  case (char) // Kiểm tra các giá trị ký tự
    5'h00: display7 = 7'b1000000; // Mã hiển thị số 0
    5'h01: display7 = 7'b1111001; // Mã hiển thị số 1
    5'h02: display7 = 7'b0100100; // Mã hiển thị số 2
    5'h03: display7 = 7'b0110000; // Mã hiển thị số 3
    5'h04: display7 = 7'b0011001; // Mã hiển thị số 4
    5'h05: display7 = 7'b0010010; // Mã hiển thị số 5

```

```

5'h06: display7 = 7'b0000010; // Mã hiển thị số 6
5'h07: display7 = 7'b1111000; // Mã hiển thị số 7
5'h08: display7 = 7'b0000000; // Mã hiển thị số 8
5'h09: display7 = 7'b0010000; // Mã hiển thị số 9
5'h0A: display7 = 7'b0001000; // Mã hiển thị chữ A
5'h0B: display7 = 7'b0000011; // Mã hiển thị chữ b
5'h0C: display7 = 7'b0100111; // Mã hiển thị chữ c
5'h0D: display7 = 7'b0100001; // Mã hiển thị chữ d
5'h0E: display7 = 7'b0000110; // Mã hiển thị chữ E
5'h0F: display7 = 7'b0001110; // Mã hiển thị chữ F
5'h10: display7 = 7'b0001001; // Mã hiển thị chữ H
5'h11: display7 = 7'b1000111; // Mã hiển thị chữ L
5'h12: display7 = 7'b1000000; // Mã hiển thị chữ O
5'h13: display7 = 7'b1100011; // Mã hiển thị chữ u
5'h14: display7 = 7'b0010010; // Mã hiển thị chữ S
default: display7 = 7'b1111111; // Mặc định tắt LED
endcase // Kết thúc khối case
endfunction // Kết thúc hàm display7

always @(*) begin // Khởi logic tổ hợp cập nhật LED
    HEX0 = 7'b1111111; HEX1 = 7'b1111111; HEX2 = 7'b1111111; // Tắt HEX0,1,2
    HEX3 = 7'b1111111; HEX4 = 7'b1111111; HEX5 = 7'b1111111; // Tắt HEX3,4,5
    case (state) // Kiểm tra trạng thái FSM hiện tại
        2'b00: begin // ST1: Trạng thái bắt đầu/chờ
            if (OP == 0) begin // Nếu OP là 0 (ADD)
                if (Led == 0) {HEX4, HEX3, HEX2, HEX1, HEX0} = {display7(5'h10), display7(5'h0E), display7(5'h11),
display7(5'h11), display7(5'h12)}; // Hiển thị HELLO
                else {HEX3, HEX2, HEX1, HEX0} = {display7(5'h07), display7(5'h01), display7(5'h03), display7(5'h05)}; //
Hiển thị MSSV
            end else if (result_valid) begin // Nếu OP là 1, hiển thị kết quả
                HEX0 = display7({1'b0, result}); // Xuất kết quả ra HEX0
            end
        end
        2'b01: begin // ST2: Trạng thái nạp dữ liệu
            HEX2 = display7({1'b0, data_b}); // Hiển thị B lên HEX2
            HEX0 = display7({1'b0, data_a}); // Hiển thị A lên HEX0
        end
        2'b10: begin // ST3: Trạng thái chọn phép toán
            if (OP == 0) {HEX2, HEX1, HEX0} = {display7(5'h0A), display7(5'h0D), display7(5'h0D)}; // Hiển thị ADD
            else {HEX2, HEX1, HEX0} = {display7(5'h14), display7(5'h13), display7(5'h0B)}; // Hiển thị SUB
        end
        2'b11: begin // ST4: Tính toán và hiển thị
            if (st4_result == 0) begin // Nếu chưa tính toán
                HEX4 = (OP == 0) ? display7(5'h0A) : display7(5'h14); // Hiển thị A hoặc S
                HEX2 = display7({1'b0, data_b}); // Hiển thị B
                HEX0 = display7({1'b0, data_a}); // Hiển thị A
            end else begin // Nếu đã có kết quả
                HEX0 = display7({1'b0, result}); // Hiển thị kết quả
            end
        end
    endcase
end
endmodule

```

Thiết kế sử dụng 5 module gồm:

- FSM_Top:
 - Định nghĩa các tín hiệu trung gian để kết nối giữa các module con: w_led (xung chậm từ bộ chia clock), w_key1_out/w_key2_out (tín hiệu nút nhấn đã sạch), dữ liệu (w_a, w_b) và các cờ trạng thái (w_valid, w_st4_result).
 - Kết nối với hệ thống: Khởi tạo Clock_Divider (u_clk): Tạo xung nhịp chậm điều khiển LED đơn (LEDR). Khởi tạo DEBOUNCER (u_btn): Lọc nhiễu cho KEY[1] và KEY[2]. Khởi tạo FSM_Controller (u_fsm): Điều khiển luồng tính toán, nhận đầu vào từ SW và KEY. Khởi tạo LED_7 (u_disp): Giải mã và hiển thị thông tin lên 6 cụm LED 7 đoạn (HEX0-HEX5) dựa trên trạng thái hiện tại của FSM và kết quả tính toán.
 - Gán trực tiếp tín hiệu w_led cho toàn bộ 10 đèn LEDR để tạo hiệu ứng nhấp nháy báo hiệu hệ thống đang hoạt động.
- Clock_Divider: Chuyển đổi tần số xung nhịp hệ thống (50 MHz) thành tần số thấp hơn để điều khiển LED.
 - Sử dụng biến counter 25-bit để tạo trễ thời gian. Với xung 50 MHz, để tạo chu kỳ 1 giây (nháy 0.5s/0.5s), bộ đếm cần chạy đến giá trị 25,000,000 (tương ứng với giá trị 24_999_999).
 - Tại mỗi cạnh lên của CLOCK_50, bộ đếm tăng dần. Khi counter đạt ngưỡng 24_999_999, hệ thống thực hiện reset counter về 0 và đảo trạng thái của Led (Led <= ~Led). Tín hiệu reset (KEY[0]) có mức ưu tiên cao nhất, đưa cả bộ đếm và trạng thái LED về mức 0 ngay lập tức khi được kích hoạt.
- DEBOUNCER: Sử dụng các thanh ghi trung gian (s0, s1, old) tạo thành một thanh ghi dịch để đồng bộ hóa tín hiệu với xung clock 50 MHz. Điều này giúp loại bỏ các xung nhiễu ngắn hạn không mong muốn. Logic assign Key1_out = k1_old & ~k1_s1 được sử dụng để phát hiện cạnh xuống (chuyển từ mức 1 sang mức 0). Nhờ việc kiểm tra trạng thái của k1_old (trạng thái trước đó) và ~k1_s1 (trạng thái hiện tại đã đảo), tín hiệu đầu ra Key1_out sẽ chỉ duy trì mức 1 trong đúng 1 chu kỳ clock tại thời điểm nhấn nút, giúp FSM nhận diện lệnh nhấn nút.
- FSM_Controller:
 - ST1 (Idle): Trạng thái nghỉ, chờ nhấn Key1 để bắt đầu.
 - ST2 (Input): Đọc giá trị từ SW[3:0] (số A) và SW[7:4] (số B). Nhấn Key2 để lưu dữ liệu.
 - ST3 (Op Selection): Chọn toán tử SW[8] (cộng hoặc trừ). Nhấn Key2 để lưu toán tử.
 - ST4 (Calculate): Thực hiện tính toán. Nhấn Key2 để thực thi phép tính và bật cờ valid/ST4_result.
 - Sử dụng reset bất đồng bộ (negedge KEY[0]). Khi reset, toàn bộ các biến A, B, OP, result và trạng thái state đều bị xóa về 0, đảm bảo hệ thống quay lại trạng thái ban đầu một cách an toàn.
 - Sử dụng toán tử điều kiện (OP == 0) ? (A + B) : (A - B) để thực hiện phép cộng hoặc trừ.
- LED_7:
 - Sử dụng hàm display7 với cấu trúc case, định nghĩa sẵn mã 7-bit cho từng ký tự (số 0-9 và chữ cái A, b, c, d, E, F, H, L, O, u, S). Các mã này điều khiển trực tiếp các thanh đoạn (a-g) trên đèn LED.
 - Khởi always @(*) kiểm tra trạng thái state nhận từ FSM để hiển thị thông tin tương ứng:
 - ST1: Chớp tắt chữ "HELLO" hoặc hiển thị kết quả nếu đã tính xong.
 - ST2: Hiển thị giá trị đang nạp của data_a và data_b.

ST3: Hiển thị "ADD" hoặc "SUB" để người dùng xác nhận phép tính.

ST4: Hiển thị phép toán đang thực hiện hoặc kết quả cuối cùng.

Hình ảnh



Hình 1.1. $OP = 0$, hiển thị "HELLO"



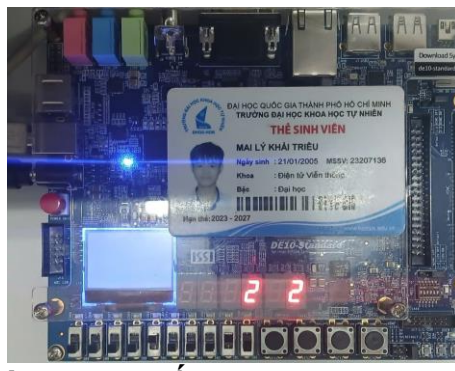
Hình 1.2. $OP = 0$, sau khi hiển thị "HELLO", hiển thị 4 số 7136, đổi xen kẽ mỗi chu kỳ clock



Hình 1.3. Nhấn KEY[1] chuyển sang ST2



Hình 1.4. Tại ST2, chọn $B = SW[7:4] = 0010$ và $A = SW[3:0] = 0010$



Hình 1.5. Nhấn KEY[2] → lưu giá trị → $A = 2, B = 2$



Hình 1.6. Nhấn KEY[1] → chuyển sang ST3, $OP = 0$ → hiện Add



Hình 1.7. Nhấn KEY[2], lưu $OP = 0$



Hình 1.8. Nhấn KEY[1] → chuyển sang ST4, hiển thị A 2 2 (Add 2 và 2)



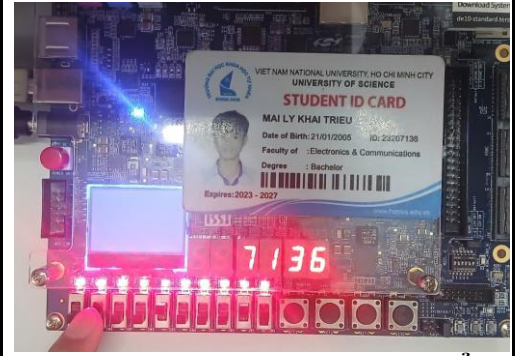
Hình 1.9. Nhấn KEY[2] để tính



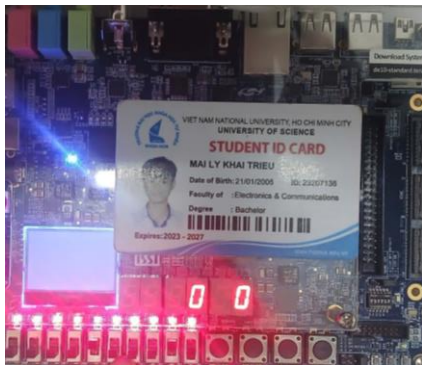
Hình 1.10. Sau khi nhấn KEY[0] để reset, bật OP = 1, không hiển thị vì đã xóa kết quả trước đó



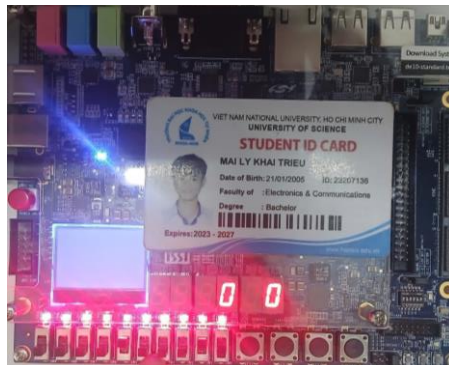
Hình 1.11. OP = 0, hiển thị "HELLO"



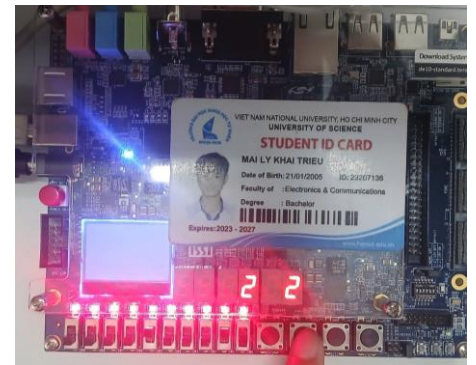
Hình 1.12. OP = 0, sau khi hiển thị "HELLO", hiển thị 4 số 7136, đổi xen kẽ mỗi chu kỳ clock (theo LEDR)



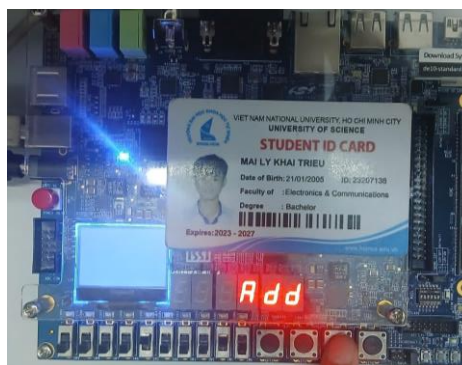
Hình 1.13. Nhấn KEY[1] chuyển sang ST2



Hình 1.14. Tại ST2, chọn B=SW[7:4]= 0010 và A=SW[3:0]= 0010



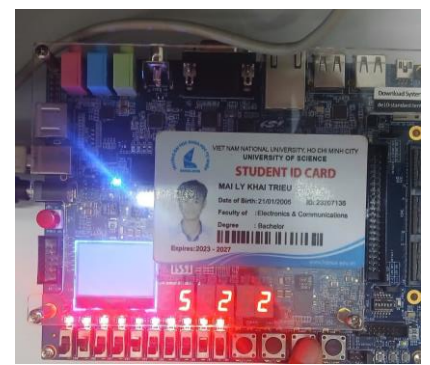
Hình 1.15. Nhấn KEY[2] → lưu giá trị → A = 2, B=2



Hình 1.16. Nhấn KEY[1] → chuyển sang ST3, OP = 0 → hiện Add



Hình 1.17. Bật OP = 1, nhấn KEY[2], lưu OP=1 vào thanh ghi hệ thống, hiện Sub



Hình 1.18. Nhấn KEY[1] → chuyển sang ST4, hiển thị S 2 2 (Sub 2 và 2)



Hình 1.19. Nhấn KEY[2] để tính



Hình 1.20. OP = 0, Nhấn KEY[1] quay về ST1, hiện HELLO và 7136 xen kẽ theo LEDR



Hình 1.21. OP = 1, hiện 0 là kết quả phép tính trước đó

Câu 2: Thiết kế một mạch đếm/bấm giờ hiển thị chục giây, giây và một phần mười giây ra cụm LED 7 đoạn (“00.0” – không cần thể hiện dấu). Sử dụng clock 50 MHz từ board FPGA, tạo quy trình đếm chính xác đến 0.1 giây, giới hạn đếm là 20 giây (dừng đếm khi đạt 20 giây, hiển thị 20).

Mã nguồn

```
// Xử lý trạng thái
module stopwatch_fsm (
    input wire CLOCK_50, // Đầu vào là xung clock hệ thống 50 MHz
    input wire reset_n, // Đầu vào reset hệ thống tích cực mức thấp
    input wire pulse_10, // Đầu vào xung nhịp 10 Hz tương ứng 0.1s
    input wire start_pause_debouncer, // Đầu vào nút start/pause đã qua chống dội
    input wire reset_debouncer, // Đầu vào nút Reset đã qua chống dội
    output reg [3:0] chục_giay, // Đầu ra hiển thị số chục giây
    output reg [3:0] donvi_giay, // Đầu ra hiển thị số đơn vị giây
    output reg [3:0] phan10_giay, // Đầu ra hiển thị số phần mười giây
    output reg running // Đầu ra báo trạng thái đồng hồ đang chạy
);

// Các trạng thái của máy trạng thái
parameter IDLE = 2'd0; // Trạng thái nghỉ, chờ nhấn start
parameter RUNNING = 2'd1; // Trạng thái đang đếm thời gian
parameter PAUSED = 2'd2; // Trạng thái tạm dừng đếm
reg [1:0] state; // State 2 bit lưu trữ trạng thái hiện tại
reg [7:0] count; // Bộ đếm 8 bit đếm thời gian từ 0 đến 200 (0.0s - 20.0s) vì 2^8 > 200

// Khởi điều khiển logic theo cạnh lên clock hoặc cạnh xuống reset
always @(posedge CLOCK_50 or negedge reset_n) begin
    if (!reset_n) begin // Nếu tín hiệu reset_n ở mức thấp
        state <= IDLE; // Đưa hệ thống về trạng thái nghỉ
        count <= 8'd0; // Reset bộ đếm về 0
        running <= 1'b0; // Tắt đèn báo trạng thái chạy
    end else begin // Nếu không reset, xử lý các trạng thái

        case (state)
            IDLE: begin // Trạng thái đang nghỉ
                running <= 1'b0; // Đồng hồ không chạy
                count <= 8'd0; // Bộ đếm luôn là 0
            end
        end case
    end
end
```

```

    if (start_pause_debouncer) state <= RUNNING; // Nhấn start chuyển sang chạy
end

RUNNING: begin // Trạng thái đang đếm
    running <= 1'b1; // Bật đèn báo đồng hồ đang chạy
    if (reset_debouncer) begin // Nếu nhấn Reset khi đang chạy
        count <= 8'd0;
        state <= RUNNING; // Reset về 0 và tiếp tục đếm ngay
    end else if (start_pause_debouncer) begin // Nếu nhấn pause
        state <= PAUSED; // Chuyển sang trạng thái dừng
    end else if (pulse_10) begin // Nếu có xung 0.1s
        if (count >= 8'd200) state <= PAUSED; // Đạt 20s thì tự dừng
        else count <= count + 1'b1; // Tăng bộ đếm thêm 1 đơn vị 0.1s
    end
end

PAUSED: begin // Trạng thái tạm dừng
    running <= 1'b0; // Tắt đèn báo chạy
    if (reset_debouncer) begin // Nếu nhấn Reset khi đang dừng
        count <= 8'd0;
        state <= IDLE; // Reset về 0 và về trạng thái nghỉ
    end else if (start_pause_debouncer) begin // Nếu nhấn Resume
        if (count >= 8'd200) state <= IDLE; // Nếu đã ở 20s thì về IDLE
        else state <= RUNNING; // Tiếp tục đếm từ giá trị cũ
    end
end
endcase
end
end
// Khởi logic tổ hợp để tách giá trị đếm thành các chữ số hiển thị
always @(*) begin
    chục_giay = count / 100; // Chia lấy phần nguyên hàng chục giây
    donvi_giay = (count % 100) / 10; // Lấy phần dư chia 100, sau đó chia 10 lấy đơn vị giây
    phan10_giay = count % 10; // Chia lấy phần dư 10 để lấy phần mười giây
end
endmodule

```

// Chống dội nút nhấn

```

module DEBOUNCER (
    input  CLOCK_50, // Đầu vào là xung clock hệ thống 50 MHz
    input  Key_In,   // Tín hiệu nút nhấn làm đầu vào
    output reg Pulse_Out // Xung đầu ra là 1 xung duy nhất 1 chu kỳ clock
);
// Đồng bộ các thanh ghi để tránh hiện tượng không xác định được trạng thái
reg key_meta; // Thanh ghi trung gian lấy mẫu tín hiệu đầu vào
reg key_sync; // Thanh ghi đồng bộ hóa tín hiệu với xung clock
reg key_stable; // Lưu trạng thái nút nhấn đã được xác nhận ổn định

// Bộ đếm thời gian (20ms) để ổn định nút nhấn
reg [19:0] debounce_counter;

initial begin
    key_meta = 1'b1; // Khởi tạo trạng thái ban đầu là 1 (nhả)
    key_sync = 1'b1; // Đồng bộ hóa cùng trạng thái nhả
end

```

```

key_stable    = 1'b1; // Đặt trạng thái ổn định mặc định là nhả
debounce_counter = 20'd0; // Reset bộ đếm về 0 khi bắt đầu
Pulse_Out     = 1'b0; // Đảm bảo xung đầu ra 0 kích hoạt khi khởi động
end

always @(posedge CLOCK_50) begin // Kích hoạt khi có cạnh lên của clock hệ thống
    key_meta <= Key_In; // Lấy mẫu đầu tiên từ tín hiệu ngoài
    key_sync <= key_meta; // Lấy mẫu lần 2 để tạo sự ổn định
    Pulse_Out <= 1'b0; // Luôn reset xung đầu ra về 0 để tạo xung đơn

    // Nếu nút không thay đổi trạng thái, reset bộ đếm
    if (key_sync == key_stable) begin // Kiểm tra xem bộ đếm đã chạy đủ 20ms (ngưỡng)
        debounce_counter <= 20'd0; // Reset bộ đếm sau khi đã đạt ngưỡng
    end

    // Nếu nút nhấn thay đổi trạng thái
    else begin
        if (debounce_counter == 20'd999_999) begin
            debounce_counter <= 20'd0;

            // Phát hiện cạnh xuống (1 -> 0)
            if ((key_stable == 1'b1) && (key_sync == 1'b0)) begin
                Pulse_Out <= 1'b1; // Kích hoạt xung đầu ra
            end

            key_stable <= key_sync; // Lưu trạng thái ổn định mới
        end
        else begin
            debounce_counter <= debounce_counter + 1'b1; // Tiếp tục đếm
        end
    end
end
endmodule

```

// Chuyển đổi mã số 4 bit sang mã điều khiển LED 7 đoạn (tích cực mức thấp)

```

module LED_7 (
    input wire [3:0] number, // Đầu vào là giá trị số cần hiển thị trong khoảng từ 0 đến 9
    output reg [6:0] SevenSegment // Đầu ra là 7-bit điều khiển các đoạn của LED từ a đến g
);
    // Khối logic tổ hợp được kích hoạt bất cứ khi nào giá trị number thay đổi, cụ thể theo các trường hợp
    always @(*) begin
        case(number)
            4'd0: SevenSegment = 7'b1000000; //Hiển thị số 0
            4'd1: SevenSegment = 7'b1111001; //Hiển thị số 1
            4'd2: SevenSegment = 7'b0100100; //Hiển thị số 2
            4'd3: SevenSegment = 7'b0110000; //Hiển thị số 3
            4'd4: SevenSegment = 7'b0011001; //Hiển thị số 4
            4'd5: SevenSegment = 7'b0010010; //Hiển thị số 5
            4'd6: SevenSegment = 7'b0000010; //Hiển thị số 6
            4'd7: SevenSegment = 7'b1111000; //Hiển thị số 7
            4'd8: SevenSegment = 7'b0000000; //Hiển thị số 8
            4'd9: SevenSegment = 7'b0010000; //Hiển thị số 9

            // Trường hợp mặc định là tắt tất cả các đoạn (nếu giá trị đầu vào > 9) để tránh lỗi
        endcase
    end
endmodule

```

```

        default: SevenSegment = 7'b1111111;
    endcase
end
endmodule

```

//Top module

```

module stopwatch_top (
    input wire CLOCK_50, // Đầu vào là xung clock hệ thống 50 MHz
    input wire [1:0] KEY, // Sử dụng 2 nút nhấn vật lý: KEY[0]=Reset, KEY[1]=Start/Pause
    output wire [6:0] HEX0, HEX1, HEX2, // Đầu ra hiển thị 7 đoạn cho phân mười, đơn vị, chục
    output wire [0:0] LEDR // Đầu ra 1 đèn LED đơn báo trạng thái hoạt động
);
    wire pulse_10; // Tín hiệu xung 10 Hz (0.1 giây)
    wire reset_debouncer, start_pause_debouncer; // Tín hiệu sau khi đã qua chống dội
    wire [3:0] chuc_giay, donvi_giay, phan10_giay; // Tín hiệu lưu giá trị các chữ số đếm
    wire running; // Tín hiệu báo trạng thái đồng hồ đang chạy

    // Khởi chia xung 50MHz xuống 10Hz (để tạo nhịp 0.1s cho đồng hồ)
    reg [22:0] div_count; // Thanh ghi bộ đếm chia xung
    always @(posedge CLOCK_50) begin // Kích hoạt khi có cạnh lên của clock hệ thống
        if (div_count >= 23'd4_999_999) div_count <= 0; // Đạt ngưỡng 5 triệu thì reset về 0
        else div_count <= div_count + 1'b1; // Nếu chưa đủ thì tăng bộ đếm thêm 1
    end
    assign pulse_10 = (div_count == 0); // Tạo xung tick 1 chu kỳ khi bộ đếm bằng 0

    // Gọi module DEBOUNCER để lọc nhiễu cho nút nhấn Reset (KEY[0])
    DEBOUNCER u_deb_reset (.CLOCK_50(CLOCK_50), .Key_In(KEY[0]), .Pulse_Out(reset_debouncer));

    // Gọi module DEBOUNCER để lọc nhiễu cho nút nhấn Start/Pause (KEY[1])
    DEBOUNCER u_deb_start (.CLOCK_50(CLOCK_50), .Key_In(KEY[1]), .Pulse_Out(start_pause_debouncer));

    // Gọi module xử lý FSM chính với các tên cổng đã được đồng bộ
    stopwatch_fsm u_fsm (
        .CLOCK_50(CLOCK_50), // Nối clock hệ thống vào FSM
        .reset_n(1'b1), // Nối chân reset cứng mức cao
        .pulse_10(pulse_10), // Nối xung 0.1s vào FSM
        .start_pause_debouncer(start_pause_debouncer), // Nối tín hiệu Start/Pause đã chống dội
        .reset_debouncer(reset_debouncer), // Nối tín hiệu Reset đã chống dội
        .chuc_giay(chuc_giay), // Nối đầu ra số chục giây vào chuc_giay
        .donvi_giay(donvi_giay), // Nối đầu ra số đơn vị giây vào donvi_giay
        .phan10_giay(phan10_giay), // Nối đầu ra số phần mười giây vào phan10_giay
        .running(running) // Nối trạng thái chạy vào running
    );

    // Gọi module giải mã LED_7 để hiển thị giá trị lên HEX0 (phần mười giây)
    LED_7 h0 (.number(phan10_giay), .SevenSegment(HEX0));
    // Gọi module giải mã LED_7 để hiển thị giá trị lên HEX1 (đơn vị giây)
    LED_7 h1 (.number(donvi_giay), .SevenSegment(HEX1));
    // Gọi module giải mã LED_7 để hiển thị giá trị lên HEX2 (chục giây)
    LED_7 h2 (.number(chuc_giay), .SevenSegment(HEX2));

    // Gán tín hiệu running ra đèn LED đơn để báo hiệu đồng hồ đang đếm
    assign LEDR[0] = running; // Bật đèn LED nếu trạng thái là running
endmodule

```

Hình ảnh



Hình 2.1. Trạng thái ban đầu, LEDR[0] không sáng vì KEY[1] không được nhấn.



Hình 2.2. Khi nhấn KEY[1], LEDR[0] sáng, đồng hồ bắt đầu đếm với 1/10 giây xuất hiện trước.



Hình 2.3. Khi 1/10 giây đếm qua 9, giây hiện 1.



Hình 2.4. Khi 1/10 giây đếm đến 9, giây hiện 2.



Hình 2.5. Khi 1/10 giây đếm qua 9, giây tăng 1.



Hình 2.6. Đồng hồ đếm đến 20.0 giây thì ngưng đếm và giữ nguyên hiển thị



Hình 2.7. KEY[0] được nhấn, đồng hồ reset về 00.0, đồng hồ ngưng đếm



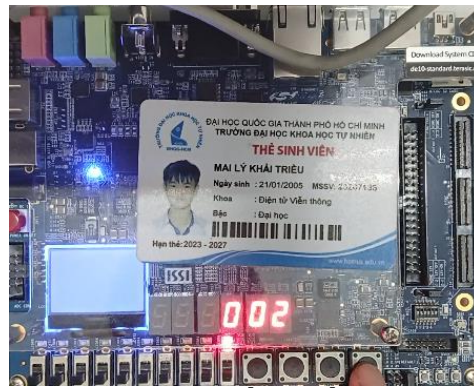
Hình 2.8. Khi KEY[1] được nhấn, đồng hồ tiếp tục đếm



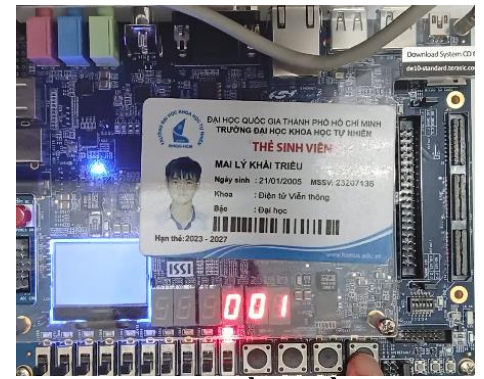
Hình 2.9. Khi KEY[1] được nhấn lần 2, đồng hồ ngưng đếm và giữ nguyên hiển thị



Hình 2.10. Khi KEY[1] được nhấn lần 3, đồng hồ tiếp tục đếm



Hình 2.11. Khi đồng hồ đang đếm, KEY[0] được nhấn để reset, đồng hồ vẫn tiếp tục đếm



Hình 2.12. Khi đồng hồ đang đếm, KEY[0] được nhấn tiếp để reset, đồng hồ vẫn tiếp tục đếm

Nhận định - giải thích:

Thiết kế sử dụng 4 module bao gồm:

- stopwatch_top:
 - Định nghĩa các chân kết nối vật lý trên board: nhận tín hiệu clock 50 MHz, hai nút nhấn (KEY0, KEY1), xuất tín hiệu ra 3 cụm LED 7 đoạn (HEX0, 1, 2) và 1 đèn LED đơn (LEDR) báo trạng thái.
 - Khai báo nội bộ để kết nối tín hiệu giữa các module con: pulse_10 mang nhịp đếm 0.1s; các biến reset_debouncer, start_pause_debouncer mang tín hiệu đã lọc nhiễu; các biến chuc_giay, donvi_giay, phan10_giay mang dữ liệu số cần hiển thị và running mang trạng thái vận hành.
 - Sử dụng bộ đếm div_count để đếm từ 0 đến 4,999,999 (tổng cộng 5 triệu nhịp clock 50 MHz).
 - Khi đạt ngưỡng, div_count quay về 0. assign pulse_10 tạo ra một xung duy nhất (logic 1) đúng tại thời điểm bộ đếm bằng 0, tạo ra tần số chính xác 10 Hz (tương ứng chu kỳ 0.1s).
 - Khởi tạo hai module DEBOUNCER riêng biệt cho KEY[0] (Reset) và KEY[1] (Start/Pause) để chống dôi nút nhấn.
 - Kết nối với module FSM (u_fsm) vào hệ thống: Truyền xung nhịp 50 MHz và xung nhịp đếm pulse_10, truyền các tín hiệu nút nhấn đã sạch từ module DEBOUNCER vào để điều khiển trạng thái, nhận về các giá trị thời gian (chuc_giay, donvi_giay, phan10_giay) và trạng thái running để xuất ra ngoại vi
 - Kết nối giá trị nhị phân (4-bit) từ bộ đếm FSM tới module giải mã LED_7 để hiển thị lên 3 cụm LED 7 đoạn: Module LED_7 (gọi tên là h0, h1, h2) sẽ nhận đầu vào là các biến thời gian tương ứng (phần mười, đơn vị, chục). Sau khi nhận giá trị (ví dụ: donvi_giay = 5), module LED_7 chuyển đổi nó thành mã 7-bit (SevenSegment) để điều khiển trực tiếp các đoạn (a-g) của đèn LED tương ứng (HEX0, HEX1, HEX2).
- DEBOUNCER:
 - Đồng bộ hóa tín hiệu bằng key_meta <= Key_In và key_sync <= key_meta để tín hiệu vật lý được chốt vào nhịp của CLOCK_50 trước khi đưa vào các khối logic khác.

- Mạch sử dụng bộ đếm để tạo khoảng thời gian chờ. Với clock 50 MHz (chu kỳ 20 ns), để tạo ra khoảng thời gian 20 ms, ta cần đếm 1 000 000 xung nhịp => 20'd999_999 chính là ngưỡng này.

- Nếu key_sync (trạng thái hiện tại) giống với key_stable (trạng thái đã xác nhận trước đó), nghĩa là nút nhấn đang ổn định. Mạch sẽ reset debounce_counter về 0. Nếu key_sync khác key_stable, nghĩa là nút nhấn vừa thay đổi trạng thái (vừa nhấn hoặc vừa nhả), bộ đếm sẽ không reset mà bắt đầu chạy để đo khoảng thời gian ổn định mới.

- Điều kiện ($\text{key_stable} == 1'b1$) && ($\text{key_sync} == 1'b0$) giúp nhận diện cạnh xuống (chuyển từ trạng thái nhả sang trạng thái nhấn). Khi phát hiện cạnh xuống sau khi đã chờ đủ 20ms ổn định, Pulse_Out được đặt lên 1 đúng trong 1 chu kỳ clock. Điều này đảm bảo FSM của bạn chỉ nhận được đúng 1 lệnh duy nhất cho mỗi lần nhấn nút, dù bạn có giữ nút đó bao lâu đi chăng nữa.

- stopwatch_fsm:

- Dùng reset bất đồng bộ, ngay khi chân reset_n ở tích cực mức thấp, toàn bộ hệ thống sẽ lập tức quay về trạng thái IDLE (ngủ), bộ đếm count bị xóa về 0, và đèn báo running tắt. Mạch này có ưu tiên cao nhất, không cần chờ nhịp clock để thực thi.
- Máy trạng thái quản lý logic chuyển đổi giữa 3 trạng thái chính:

IDLE (Trạng thái chờ): Khi đang ở , hệ thống giữ bộ đếm count ở mức 0. Chỉ khi có tín hiệu start_pause_debouncer (nhấn nút Start), máy trạng thái mới chuyển sang RUNNING

RUNNING (Trạng thái đếm): Đèn báo running được bật (1'b1). Nếu nhấn nút Reset (reset_debouncer), count về 0 nhưng vẫn giữ trạng thái RUNNING để đếm tiếp ngay lập tức. Nếu nhấn nút Start/Pause, chuyển sang PAUSED. Mỗi khi có nhịp pulse_10 (0.1 giây), bộ đếm count tăng thêm 1. Nếu count đạt tới 200 (20.0s), hệ thống tự động chuyển sang PAUSED để dừng đếm.

PAUSED (Trạng thái tạm dừng): Đèn báo running tắt. Nếu nhấn Reset, count về 0 và chuyển về IDLE (dừng hẳn, không tự đếm tiếp). Nếu nhấn Start/Pause, kiểm tra nếu thời gian chưa hết ($\text{count} < 200$) thì quay lại RUNNING để đếm tiếp từ giá trị cũ. Nếu đã hết thời gian, nó quay về IDLE.

- Chuyển đổi giá trị count (đơn vị 0.1s) thành các chữ số thập phân để hiển thị lên 3 cụm LED 7 đoạn: $\text{count} / 100$ là tách lấy hàng chục giây, $(\text{count} \% 100) / 10$ là tách lấy hàng đơn vị giây, $\text{count} \% 10$ là tách lấy hàng phần mười giây.

- LED_7: Module stopwatch_top đã khởi tạo (gọi) module LED_7 ba lần (với tên h0, h1, h2) để điều khiển tương ứng ba cụm LED 7 đoạn HEX0, HEX1, và HEX2 hiển thị giá trị thời gian. module sử dụng lệnh case để tra. Với mỗi giá trị 4-bit (từ 0 đến 9) thì đã được lập sẵn một mã 7-bit tương ứng. Mỗi bit trong chuỗi 7-bit SevenSegment điều khiển một đoạn của LED (thường theo thứ tự a, b, c, d, e, f, g). Nếu tín hiệu đầu vào number vượt quá phạm vi 0-9 (ví dụ như mã lỗi hoặc nhiễu), toàn bộ các đoạn LED sẽ được gán mức 1 (tất hết).